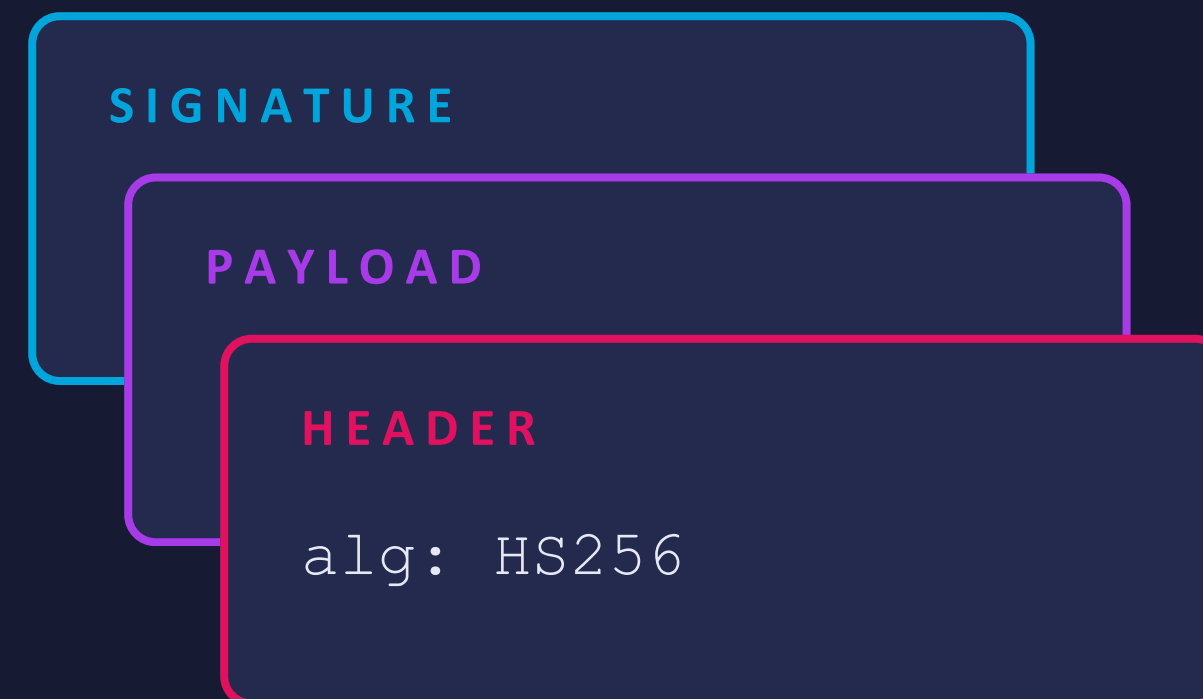




Autenticação com JWT

Spring Boot + Angular — conceito e solução desenvolvida



eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbHVubyJ9.wblW5oxViZWtzCpHE2I

Programação WEB - Jorge Lima

Táris Barreto, Ylo Bittencourt, João França, Samuel Santos e Guilherme Crocamo

HTTP não lembra quem você é

Cada requisição HTTP é independente (stateless). Depois do login, como o servidor sabe quem está chamando /api/me?



Reenviar a senha sempre

- A senha não deve acompanhar cada requisição.
- Isso aumenta a exposição da credencial e repete o processo de autenticação.



Login uma vez, depois token

- Senha só no POST /auth/login
- Servidor devolve um Access Token
- Requisições seguintes enviam só o token

Autenticação × Autorização



Autenticação

Quem é você?

- Provar a identidade (e-mail + senha)
- Acontece uma vez, no POST /auth/login
- Resultado: o JWT com o "sub" do usuário

`AuthenticationManager + BCrypt`



Autorização

O que você pode fazer?

- Decidir o acesso a cada rota
- Acontece em toda requisição
- Usa os perfis do token (ROLE_USER)

`.anyRequest().authenticated()`

Sessão no servidor × Token no cliente

	Sessão (cookie JSESSIONID)	Token JWT
Onde fica o estado	Memória / banco do servidor	Dentro do próprio token
O que o cliente envia	Cookie com o ID da sessão	Authorization: Bearer <jwt>
Vários servidores	Precisam compartilhar sessões	Qualquer um valida com a chave
SPA e mobile	Depende de cookies	Funciona com qualquer cliente HTTP
Logout / revogação	Fácil: apaga a sessão	Vale até expirar (por isso: 1 hora)

JWT por dentro

Header.Payload.Signature — três partes separadas por ponto

HEADER

```
{  
  "alg": "HS256"  
}
```

Indica o algoritmo usado na assinatura.

PAYLOAD

```
{  
  "sub": "aluno@email.com",  
  "roles": ["ROLE_USER"],  
  "exp": ...  
}
```

Claims do usuário e validade.

SIGNATURE

HMAC-SHA256(
 header + payload,
 chave secreta
)

Garante integridade e autenticidade.

Importante: Base64URL é codificação, não criptografia. O payload pode ser lido.

Por que ninguém consegue falsificar?



O fluxo completo



requisições seguintes

```
GET /api/me
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbHVub0BlbWFpbC5jb20i...
```

```
// a senha NÃO é reenviada – só o Access Token
```

Arquitetura da solução



Frontend · Angular 22

LoginComponent

tela onde o usuário coloca e-mail e senha

AuthService

faz a comunicação com o POST /auth/login, e guarda o JWT.

authInterceptor

anexa o Bearer e trata 401

authGuard

protege a rota /home, verificando se existe token válido

HomeComponent

consome /api/me



Backend · Spring Boot 4

SecurityConfig

configura regras, stateless, BCrypt, 401

AuthController

recebe POST /auth/login

JwtService

gera e valida tokens (JJWT)

JwtAuthenticationFilter

lê o Bearer a cada requisição

GlobalExceptionHandler

tratamento de erros

HTTP + JSON

Bearer JWT

:4200 → :8080

Estrutura do projeto

Spring Boot

backend/.../authapi

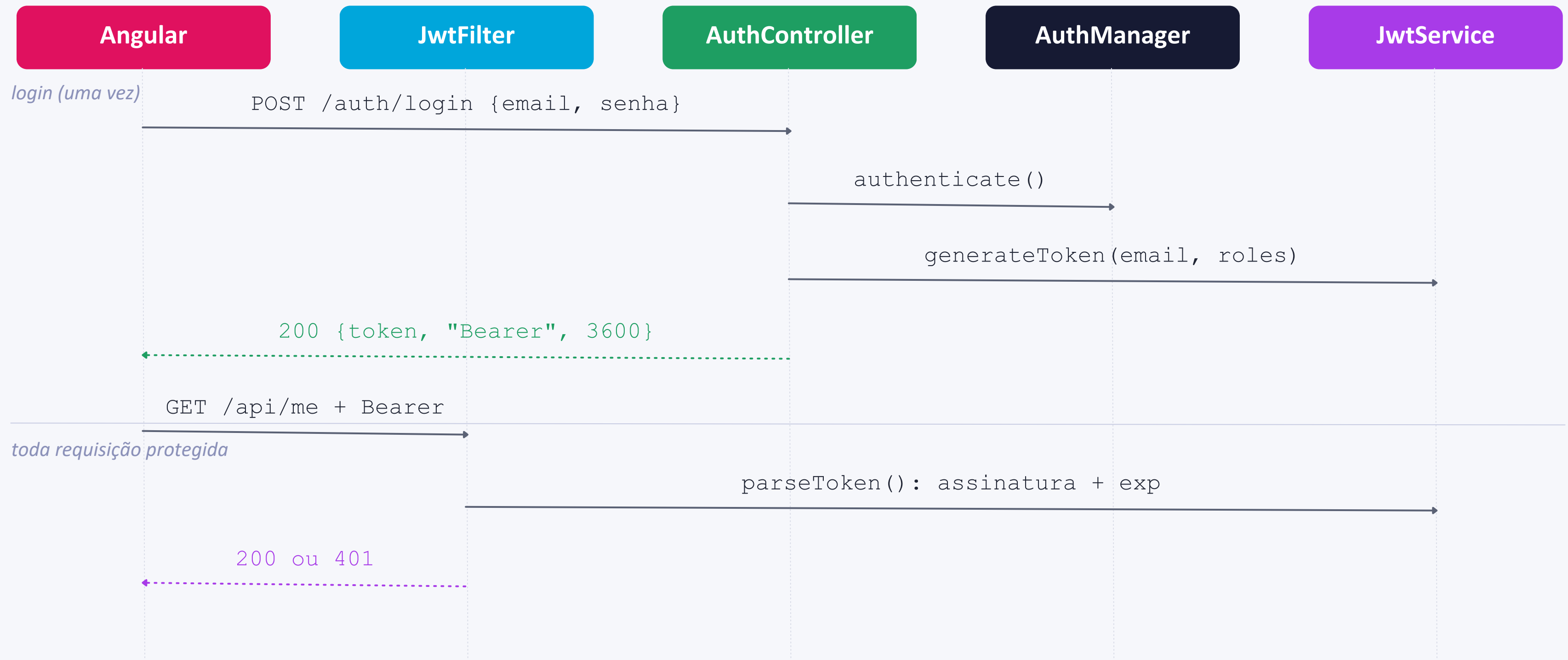
- config/
 - SecurityConfig
 - JwtAuthenticationFilter
 - CorsConfig
- controller/
 - AuthController
 - UsuarioController
- service/JwtService
- exception/
- dto/

Angular

frontend/.../src/app

- login/
- home/
- services/
 - auth.service.ts
 - auth.interceptor.ts
- guards/auth.guard.ts
- app.routes.ts
- api.config.ts

Diagrama de sequência



SecurityConfig: as regras do jogo

Regras de segurança definidas para a API

```
config/SecurityConfig.java

http
    .csrf(csrf -> csrf.disable())
    .sessionManagement(s -> s
        .sessionCreationPolicy(STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(POST, "/auth/login")
            .permitAll()
        .anyRequest().authenticated())
    .exceptionHandling(ex -> ex
        .authenticationEntryPoint(entryPoint))
    .addFilterBefore(jwtAuthenticationFilter, ...);
```

- **STATELESS**
nenhuma sessão HTTP é criada, a cada requisição, o cliente manda o JWT novamente
- **permitAll**
o login é público, já que no primeiro login não possui um JWT
- **authenticated**
qualquer outra requisição exige token
- **EntryPoint**
sem token → 401 em JSON

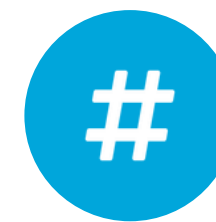
Senha protegida com BCrypt

config/SecurityConfig.java

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

usuário de demonstração

```
User.builder()
    .username("aluno@email.com")
    .password(encoder.encode("123456"))
```



Formato do hash

\$2a\$10\$N9qo8uLOickgx2ZMRZoMye
IjZAgcf17p92ldGxad68LJZdL17lhWy

\$2a\$ versão do algoritmo

10\$ custo: 2¹⁰ rodadas (lento de propósito)

salt aleatório: mesma senha, hash diferente

POST /auth/login: validando a senha

```
controller/AuthController.java

@PostMapping("/login")
public ResponseEntity<LoginResponse> login(
    @Valid @RequestBody LoginRequest req) {

    Authentication auth = authenticationManager
        .authenticate(new UsernamePassword...(
            req.email(), req.senha()));

    String token = jwtService
        .generateToken(auth.getName(), roles);
    return ok(new LoginResponse(token, "Bearer", 3600));
}
```

Senha correta · 200

```
{ "token": "eyJ...",
  "tokenType": "Bearer",
  "expiresIn": 3600 }
```

Senha errada · 401

```
{ "status": 401,
  "erro": "E-mail ou
  senha inválidos" }
```


JwtService: gerar e validar token

gerar · generateToken()

```
Jwts.builder()
    .subject(email)
    .claim("roles", roles)
    .issuedAt(agora)
    .expiration(agora + 1h)
    .signWith(key, HS256)
    .compact();
```

validar · parseToken()

```
try {
    Jwts.parser()
        .verifyWith(key)
        .build()
        .parseSignedClaims(token);
    return Optional.of(claims);
} catch (JwtException e) {
    // assinatura errada ou expirado
    return Optional.empty();
}
```

JwtAuthenticationFilter: a cada requisição



Sem token, token adulterado ou expirado?

O filtro não autentica ninguém: o Spring Security bloqueia a rota e o `AuthenticationEntryPoint` responde 401 em JSON. Nenhuma consulta a banco ou senha — tudo vem do token assinado.

Angular: service, interceptor e guard

services/auth.service.ts

```
login(email, senha) {  
  return http.post(API + "/auth/login",  
    { email, senha })  
    .pipe(tap(r => localStorage  
      .setItem("access_token", r.token)));  
}
```

guards/auth.guard.ts

```
export const authGuard = () =>  
  auth.isAuthenticated() // lê "exp"  
    ? true  
    : router.createUrlTree(["/login"]);
```

services/auth.interceptor.ts

```
export const authInterceptor =  
  (req, next) => {  
    const request = token && isApi  
      && !isLogin  
      ? req.clone({ headers: {  
        Authorization: `Bearer ${token}`  
      } })  
      : req;  
    return next(request).pipe(  
      catchError(err => {  
        if (err.status === 401) {  
          auth.logout();  
          router.navigate(["/login"]);  
        } ... }));  
  }
```


Cada situação, o status HTTP certo

Situação	Status	O que o Angular faz
Login com credenciais corretas	200	Guarda o token e vai para /home
E-mail vazio ou malformado	400	Mostra a mensagem de validação
Senha/Token Inválido	401	Volta ao login
Backend fora do ar	—	Avisa que não conseguiu conectar

Roteiro da demonstração

Acesse sua conta

E-mail

Senha

E-mail ou senha inválidos

Entrar

Usuário de teste: **aluno@email.com** / 123456

Área protegida

Você está autenticado como: aluno@email.com

E-MAIL
aluno@email.com

PERFIS
ROLE_USER

TOKEN EXPIRA EM
27/09/2026 17:43:38

ACCESS TOKEN (JWT)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWUiOjEwIjBhbHVub0B1bWVpbC5jb20iLCJyY2x1cyI6IjE3OT0xOTY3IiwiaWF0IjoxNzQ1MjM0MjE4LCJleHAiOiE3OTAxNDE4MTh9.Od-6E7cDu4hT8bkiN0LukxNKOtyMBC1InB8PSukDN0Y
```

Chamar /api/me novamente

Sair

- 1 Senha errada → erro na tela
- 2 Login → /home com dados do /api/me
- 3 F12 › Network: Authorization: Bearer
- 4 Alterar o token → 401 → login
- 5 Sair → /home bloqueada



Senha uma vez.
Token em todas as outras.

API stateless

Spring Security + JWT

Token assinado

HS256 + expiração

Angular

interceptor + guard